RX.JS



Anas Inani Marouf

DFFM - 14/10/2026

INDICE

1. ¿Qué es y para qué sirve?..... 3
2. Conceptos clave.....4
3. Requisitos e instalación..... 5
- Requisitos..... 5
- Comandos de instalación de RxJs paso a paso..... 8
- Estructura de carpetas generada, y explicación de los archivos importantes..... 10
- Demostracion del funcionamiento:..... 11

4. Ejemplo básico de uso..... 14
- Funcionamiento:.....15
- Fragmentos del codigo importantes:.....15

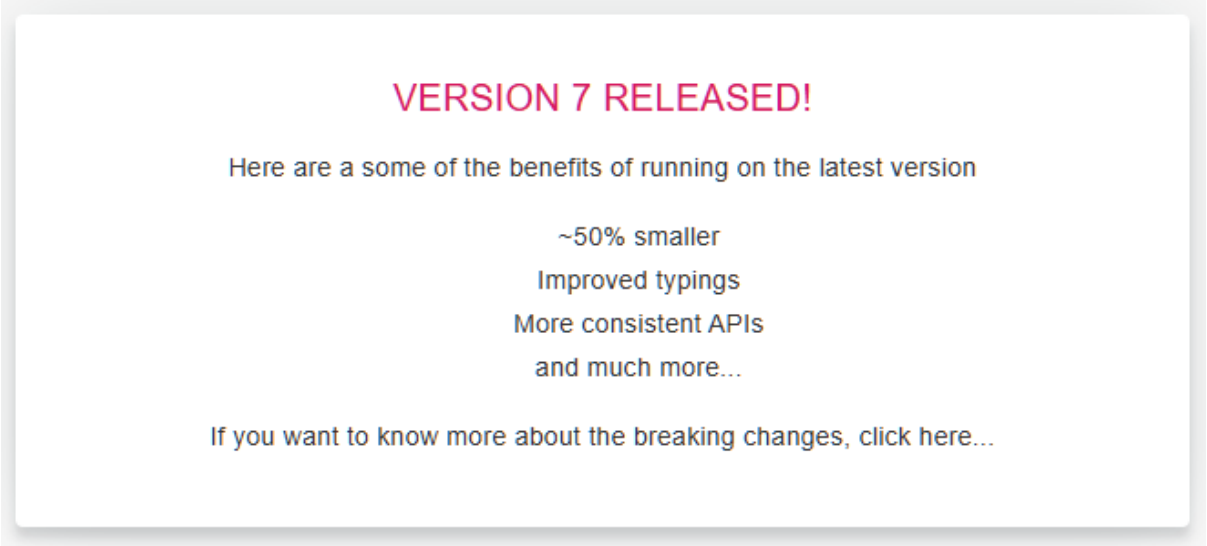
5. Ventajas, inconvenientes y conclusion..... 18
6. Recursos y bibliografia..... 19

1. ¿Qué es y para qué sirve?

Es una libreria para javascript, que se usa para la programación reactiva mediante el uso de ‘observables’ y ‘observadores’. Basicamente como funciona es, hay un observador que espera a que se introduzca uno o varios valores, y cuando se ejecuta el observable, el observador recibe la informacion introducida y se hace con ellos lo que yo quiera.

El proposito de esta tecnologia, como el de cualquier otro, es escribir en menos lineas de codigo lo que hubieras escrito en javascript con muchas mas, o sea, ahorrar tiempo. Pero no es solo ahorrarte tiempo escribiendo codigo, esque ademas realiza tareas muy complejas en cuestion de palabras, por ejemplo, en javascript nativo, para que se empiezen ha hacer cambios despues de un determinado tiempo desde que el usuario ha escrito unos datos, 500 milisegundos por ejemplo, tendrias que liarla mucho para conseguirlo, y en javascript ya hay metodos que lo hacen, pero son super ineficientes, sin embargo, con esta libreria, solo tienes que llamar al metodo ‘*debounceTime(300)*’ y ya estaria, no es solo mas rapido de escribir sino que ademas es mucho mas eficiente.

Esta tecnologia nacio de un hombre llamado Erik Meijer, mientras trabajaba en microsoft. Esto lo estaban haciendo al principio para C#, pero como vieron que era muy util lo llevaron tambien a otros lenguajes. Ahora mismo, nadie es propietario de esta libreria, pero es open source, aunque hay gente que va mejorandola cada dia, como por ejemplo Ben Lesh y su equipo. Actualmente, la version mas reciente es la 7.8.2, que tiene significantes mejoras con respecto a las anteriores versiones:



2. Conceptos clave

La propia página de RxJs tiene un pequeño apartado de conceptos esenciales que hay que entender para el manejo de esta tecnologia, asi que esas son las que voy a explicar en este apartado:

a. Como ya dije antes, el '**observable**' es un metodo que esta a la espera de nuevos cambios, y una vez reconozca que se estan haciendo cambios, los envia al codigo, pero esto no tiene ningun fin por si solo, es por eso que estan los llamados '*observadores*' que es el siguiente concepto fundamental

b. Un '**observer**' u *observador* es una coleccion de callbacks (un callback es cuando a una funcion le pasas otra funcion como argumento), solo que en vez de enviarle una funcion al *observable*, le mandas directamente un *observador*. Esto lo que hace es que cada vez que el observable se ejecuta, el *observador* tambien lo hace, y los datos que haya pasado el *observable*, lo recibe el *observador*, y ya tu decides lo que se hacen con esos datos

c. Todo esto de los observables y observadores esta muy bien, pero que pasa si el usuario cancela un movimiento que se estaba ejecutando, por ejemplo, el de cancelar una subscripcion a un canal de youtube al que acabas de pulsar? Normalmente, se tendria que esperar a que terminase todo lo necesario para la suscripción, y una vez terminado, ya se desuscribiria. Bueno pues en RxJs no pasa, ya que hay un objeto muy util que resuelve este problema en especifico, el '**subscription**' , y en vez de esperar a que todo el proceso acabe, lo que hace es desechar toda la ejecucion del observable, en este caso, la subscripcion a ese canal de youtube

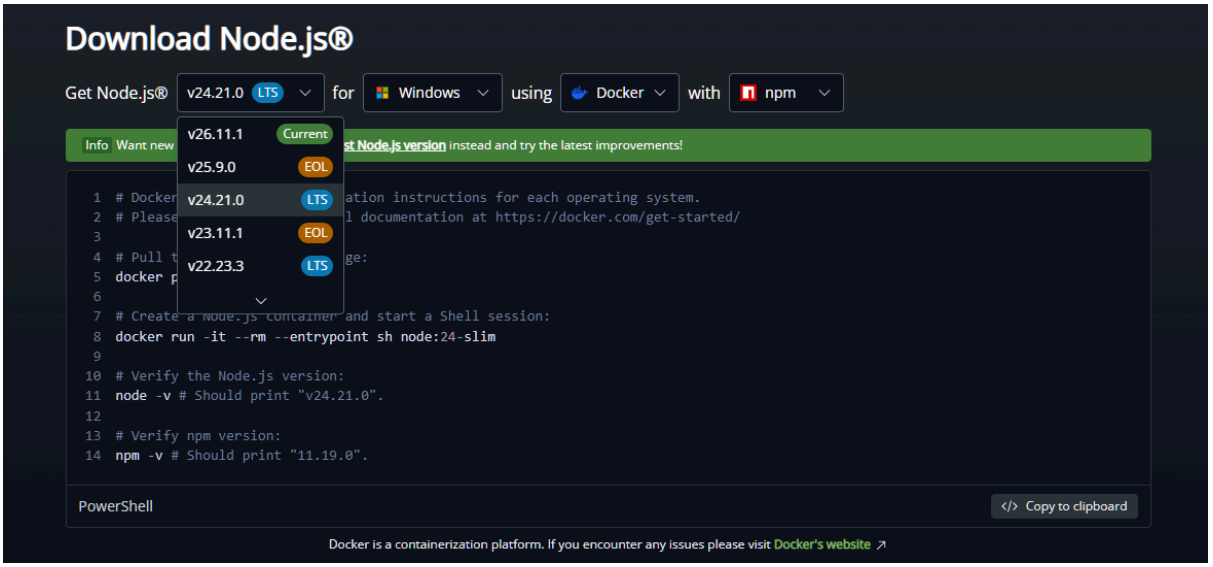
d. Y ahora, lo que mas util hace RxJs segun los desarrolladores, son los '**operators**' u operadores. Aunque los observadores son la base, los operadores son los que permiten que esta tecnologia haga en tan solo un par de lineas de codigo, lo que la asincronia compleja tomaria en muchas

e. Normalmente, un observador siempre va con un observador, pero claro, que pasa si yo quiero subscribirme a varios canales de youtube a la vez en vez de uno solo, tomaria mucho tiempo y recursos para hacer una tarea que realmente se esta repitiendo. Pues el '**subject**' o sujeto resuelve este problema. Este objeto es equivalente a un *observable*, solo que en vez de solo poder tener un observador, puede tener múltiples, agilizando las tareas que requieren mas de una resolucion para una misma accion

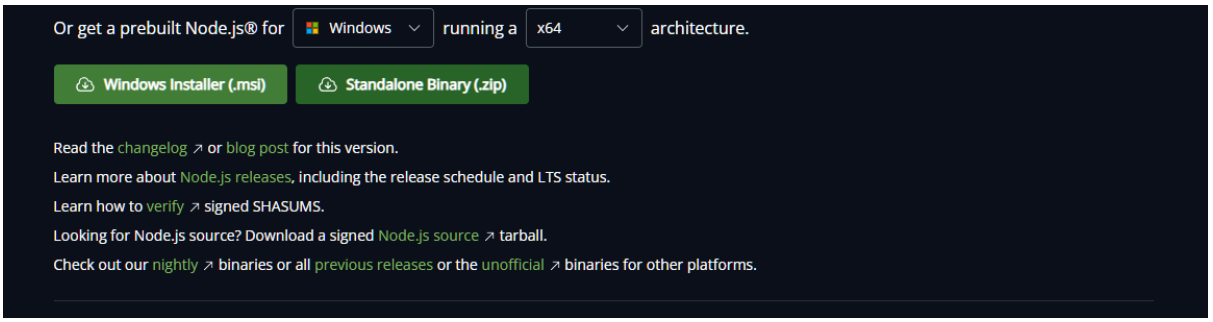
f. Por ultimo, estan los '**schedulers**' u horarios. Estos basicamente son organizadores que controlan cuando se ejecutan las operaciones y cuando un *observador* recibe una notificación del *observable*

3. Requisitos e instalación

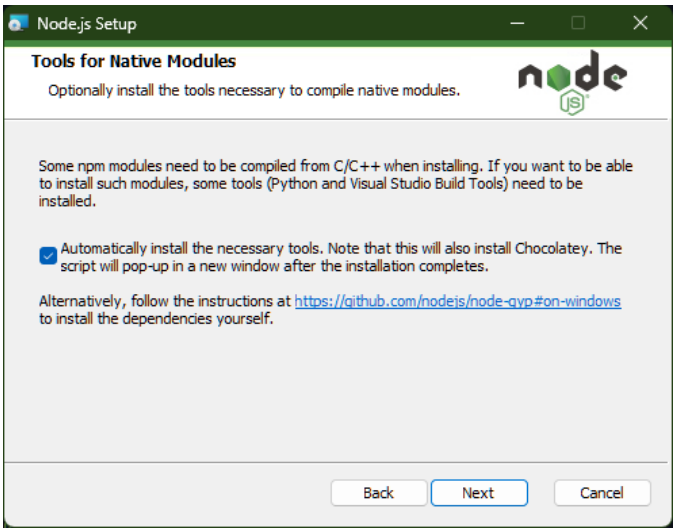
- **Requisitos**.
    - Editor: Da completamente igual, se podria usar incluso con el bloc de notas. Pero en mi caso, voy a usar Visual Studio Code, que creo que es la mas reconocida y comoda de usar
    - Paquetes: Antes de instalar RxJs, hay que tener instalados tanto node.js como npm. Para instalar node.js hay que visitar la pagina NodeJs. En el apartado de versiones, es preferible seleccionar la mas reciente y que sea LTS, porque son las que mas soporte van a recibir a lo largo de los años



- Lo podemos instalar copiando lo que aparece en el cuadro y pegandolo en powershell, o bien dandole al boton de abajo

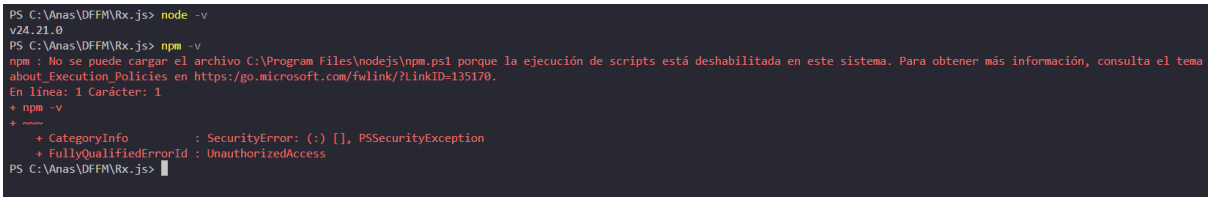


- En mi caso, simplemente voy a descargar el archivo .msi y lo voy a ejecutar. Le damos todo a next y cuando aparezca el siguiente mensaje, le damos a automáticamente instalar las herramientas necesarias, y le damos todo a next

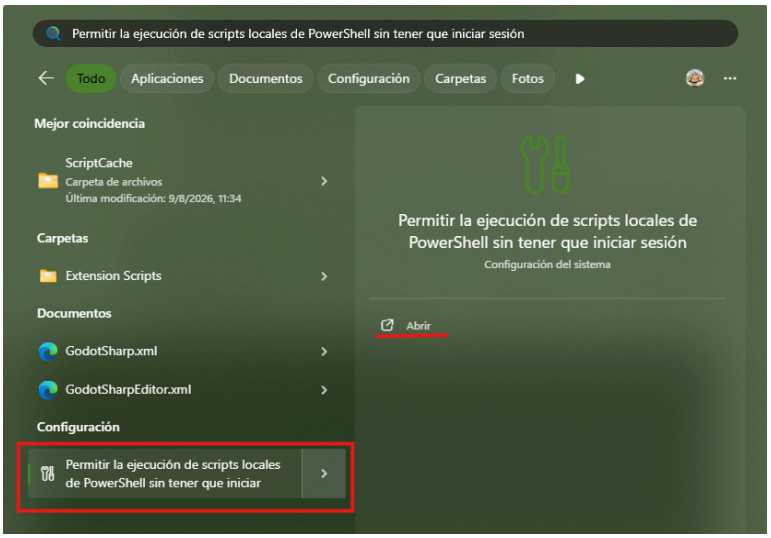


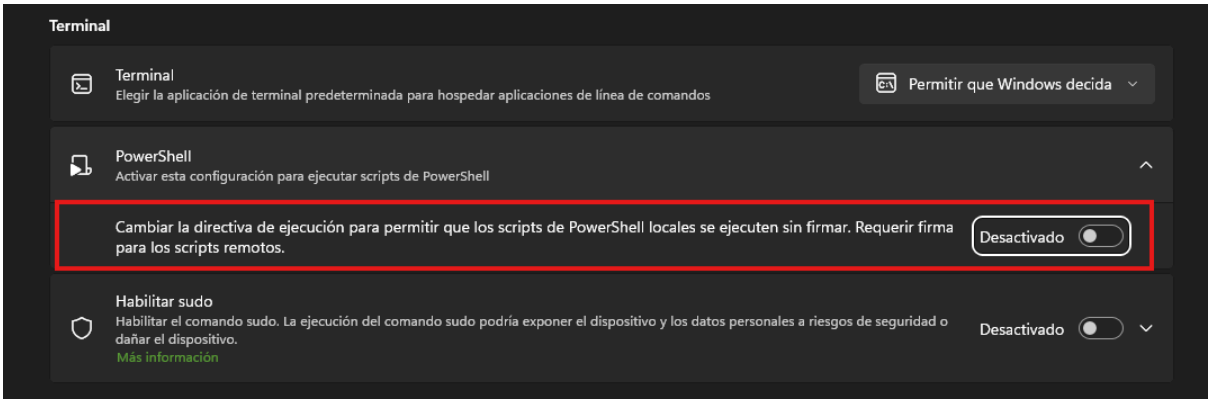
- Una vez instalado, se te abrirá una ventana de powershell, simplemente le tienes que dar a enter dos veces y ya se te instalara todo lo necesario

- Se puede ver si lo tienes instalado y que version con los siguientes comandos: '**node -v**' y '**npm -v**'

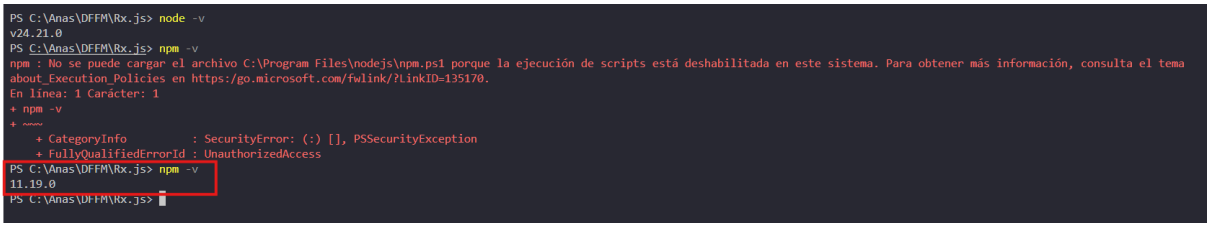


- Si como a mi te da un error en el npm, simplemente buscas en el buscador de windows lo que viene abajo y activas la opcion.

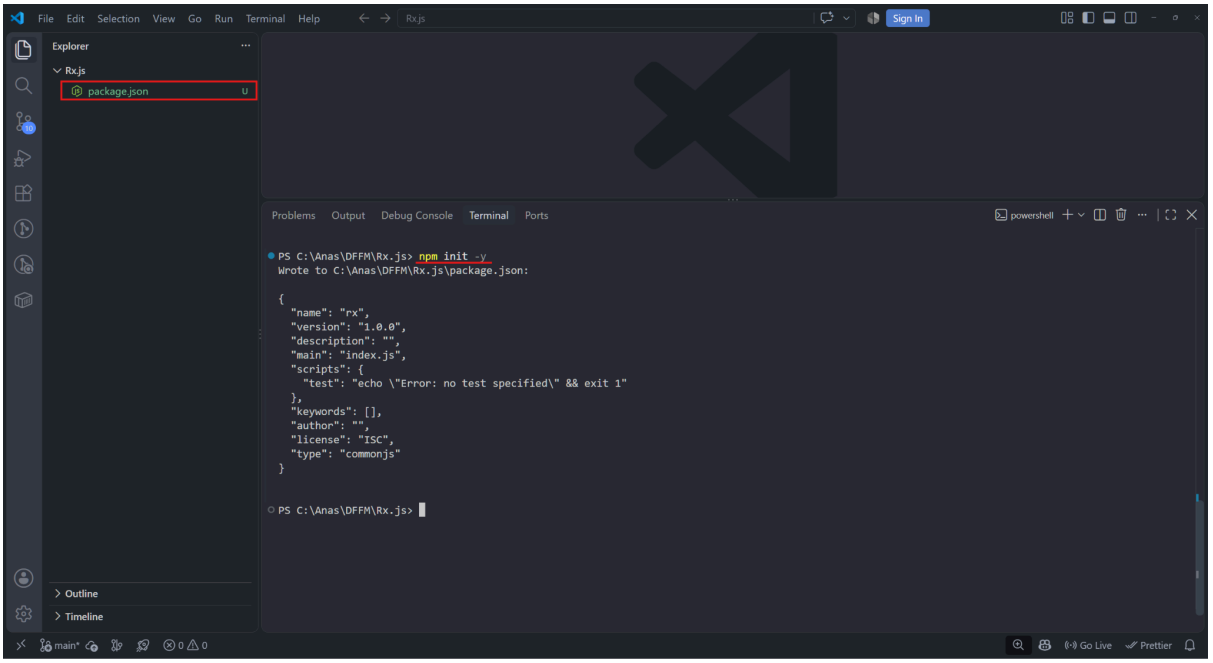




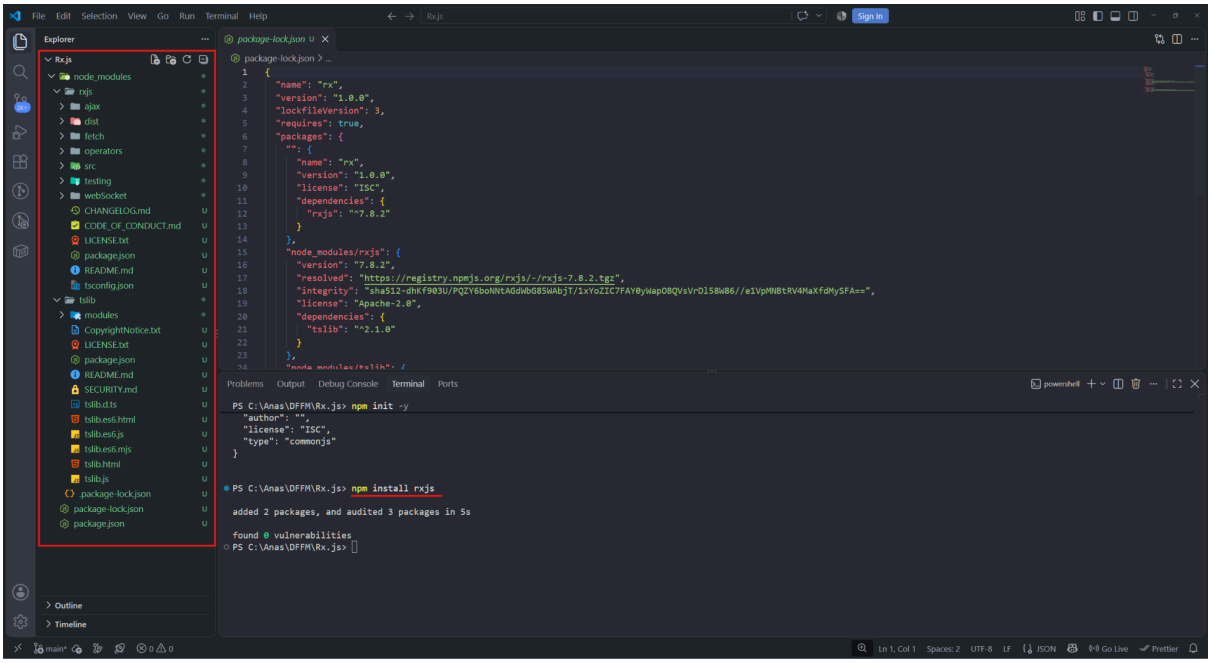
- Y ahora si vuelves a ejecutar el comando de antes te aparecera la version:



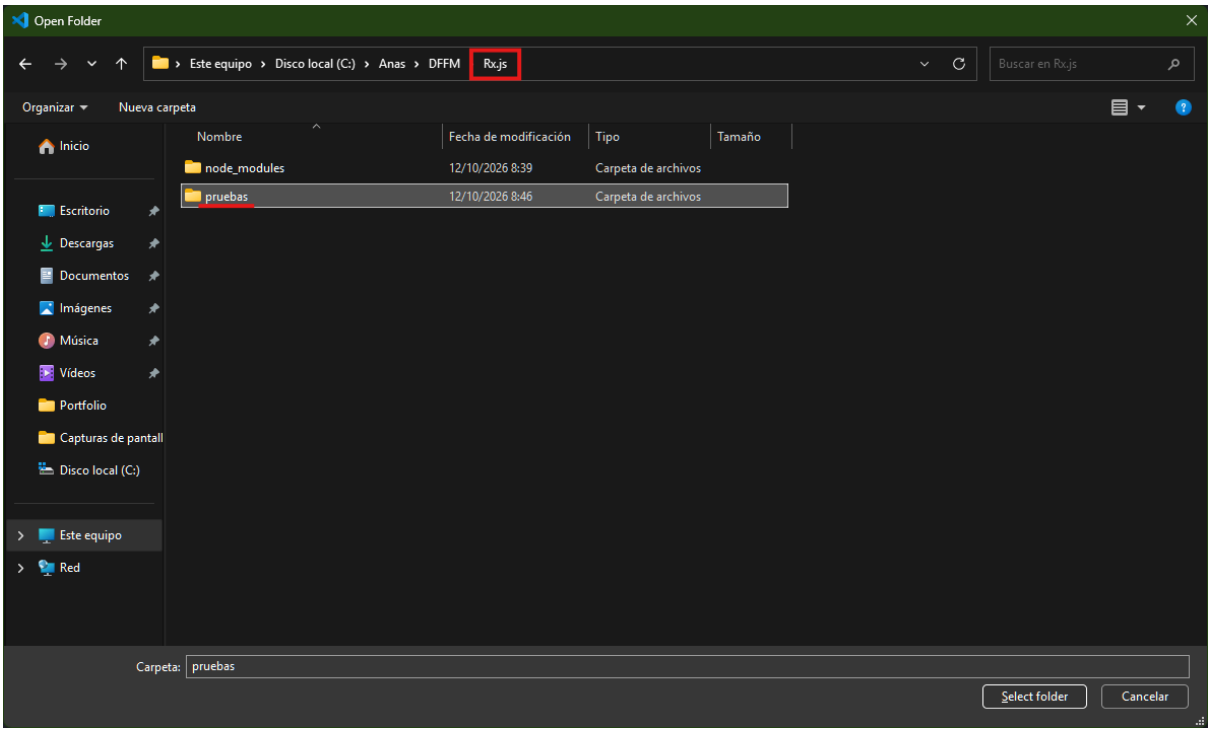
- Comandos de **instalación** de RxJs paso a paso
    - Hay que tener un proyecto inicializado, si no lo tienes, simplemente pon este comando en la terminal: '**npm init -y**'



    - Lo siguiente es poner el siguiente comando, para generar la estructura de carpetas necesaria: '**npm install rxjs**'

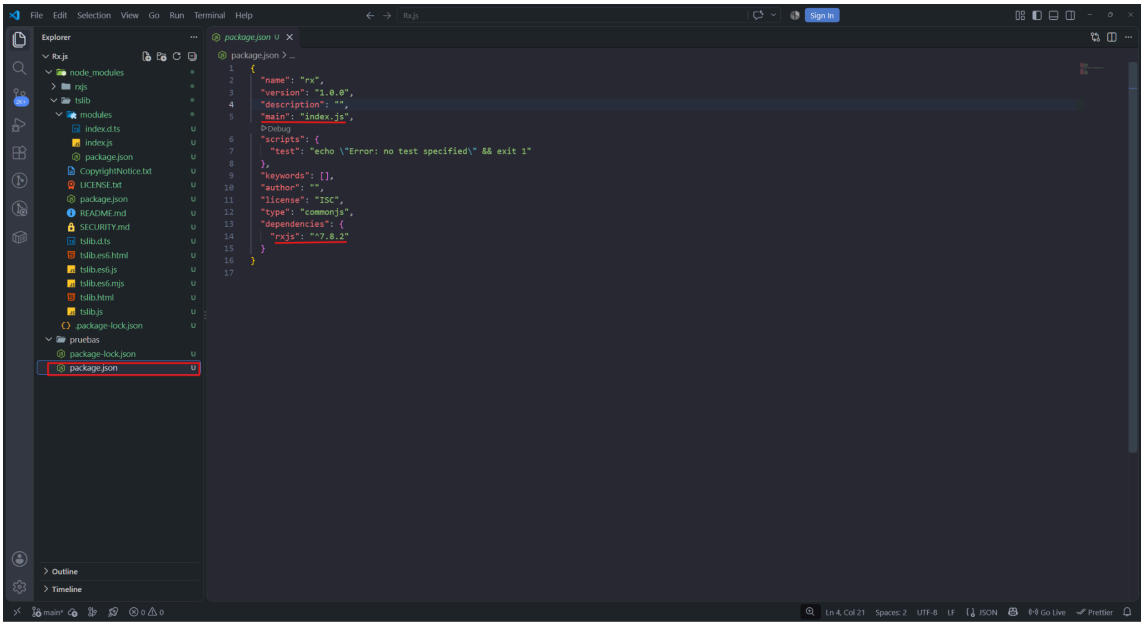


- Ya solo queda crear la carpeta necesaria para empezar con el proyecto, y es muy importante que el proyecto lo crees dentro de la carpeta en donde has instalado el RxJs, como se muestra en la imagen, tiene que quedar al mismo nivel que node_modules. Y este paso es importante, porque si lo creas por ejemplo en el escritorio, no vas a poder importar esta herramienta directamente en tu proyecto

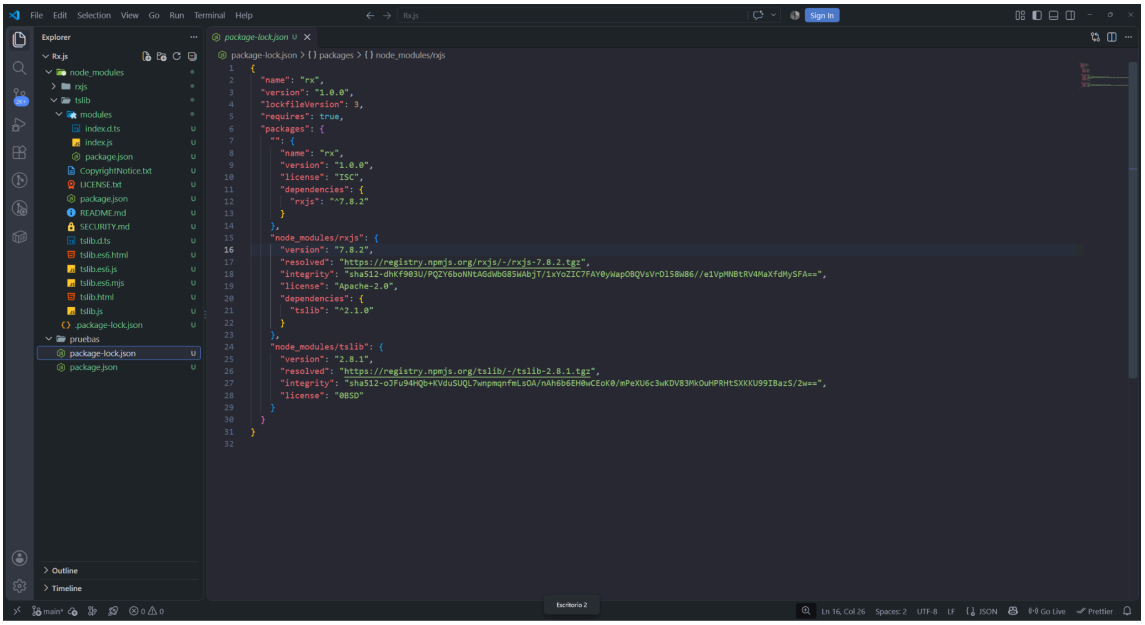


- **<u>Estructura de carpetas</u>** generada, y explicación de los **<u>archivos importantes</u>**.

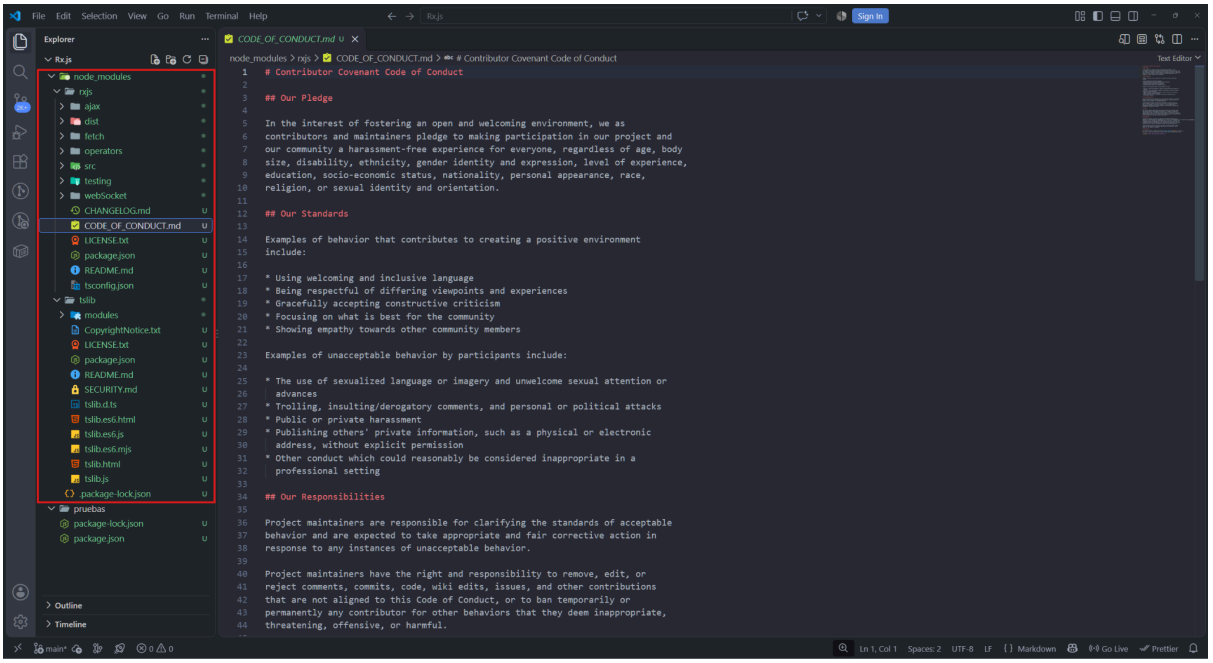
    - ***package.json***: Dentro de este archivo podemos ver la version, y ademas el archivo principal donde se ejecutara la aplicacion. Como lo que voy a hacer yo es una prueba, no es necesario, pero si tuvieras una aplicacion ya desarrollada, es importante que cambies la ruta por el archivo principal de tu proyecto



    - ***package-lock.json***: Este archivo sirve para decirle a node_modules que paquetes necesita para funcionar, en este caso rxjs y tslib (que es la libreria de typescript, porque rxjs esta basada en el)



- ***node_modules***: Es la carpeta principal en donde estan almacenadas todas las librerias almacenadas. Importante no tocarlo, si no podrias desconfigurar todo el proyecto, asi que de momento nos olvidamos de el



- Demostracion del **funcionamiento**:

  - Rapido inciso. Para importar el rxjs y todo lo que necesitemos, lo comun antes era importarlo mediante constantes, el problema esque no es lo mas habitual actualmente, ya que existe otra forma de importarlo, y que no solo es mas rapido (por el hecho de ser asincrono) si no que ademas es el estandar de javascript, y es el que se utiliza para las paginas web de ahora

```
const { of } = require('rxjs');
const { map } = require('rxjs/operators');
```

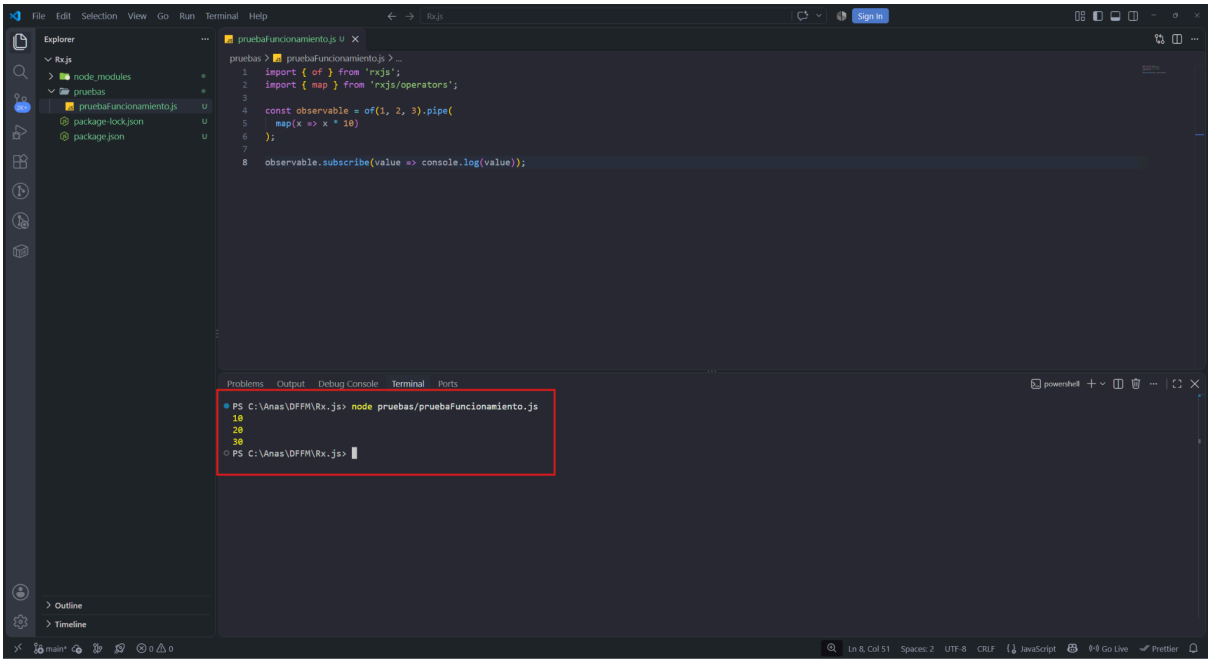
```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';
```

  - En lugar de esto ⬆️ , la mejor forma es la esta ⬆️ , pero para poder utilizarla tenemos que irnos al archivo package.json y cambiar el tipo de “commonjs” a module:

```
package.json > ...
{
  "name": "rx",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "rxjs": "^7.8.2"
  }
}
```

```
package.json > abc type
{
  "name": "rx",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "rxjs": "^7.8.2"
  }
}
```

- Y ahora si ejecutamos un ejemplo rapido, podemos ver que todo funciona correctamente:



- Otro pequeño inciso, en mi caso como quiero hacer un ejemplo de prueba en el navegador, para que se vea visualmente, tengo que o bien utilizar la importacion mediante urls:

```
import { fromEvent } from 'https://cdn.jsdelivr.net/npm/rxjs@7.8.2/+esm';
```

- O bien instalar vite (‘npm install vite --save-dev’ es lo mismo que ‘npm install vite -D’:

```
PS C:\Anas\DFFM\Rx.js> npm install vite --save-dev

added 15 packages, and audited 18 packages in 16s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Anas\DFFM\Rx.js>
```

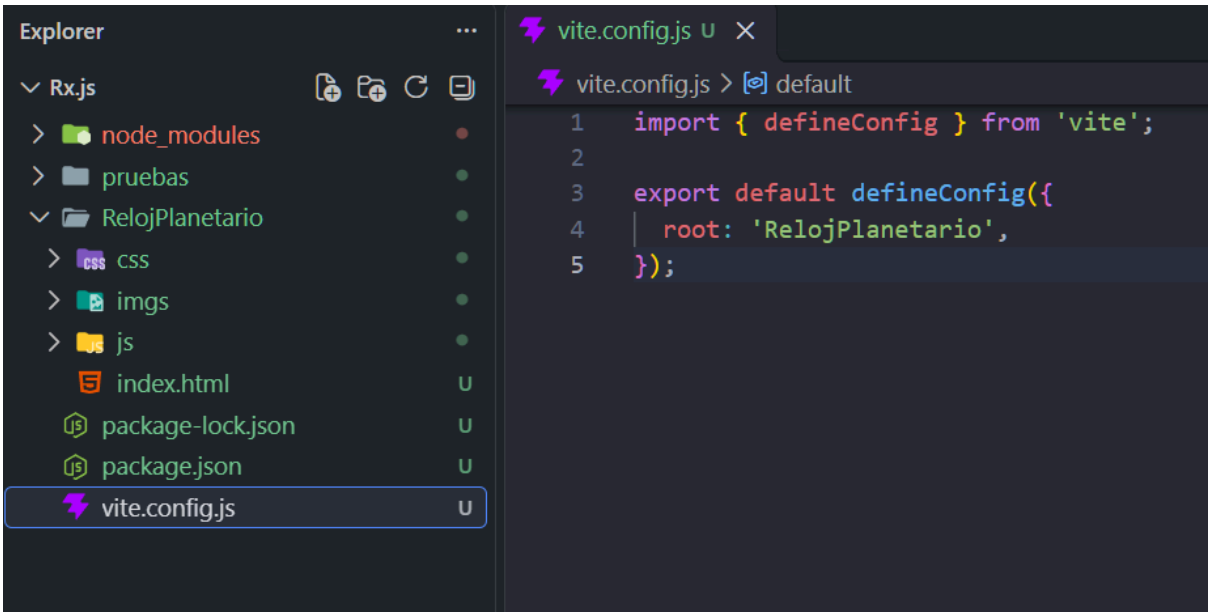
- Despues, crear un archivo ‘vite.config.js’ en la carpeta raiz (donde hayas instalado RxJs), y poner lo siguiente:

```
import { defineConfig } from 'vite';

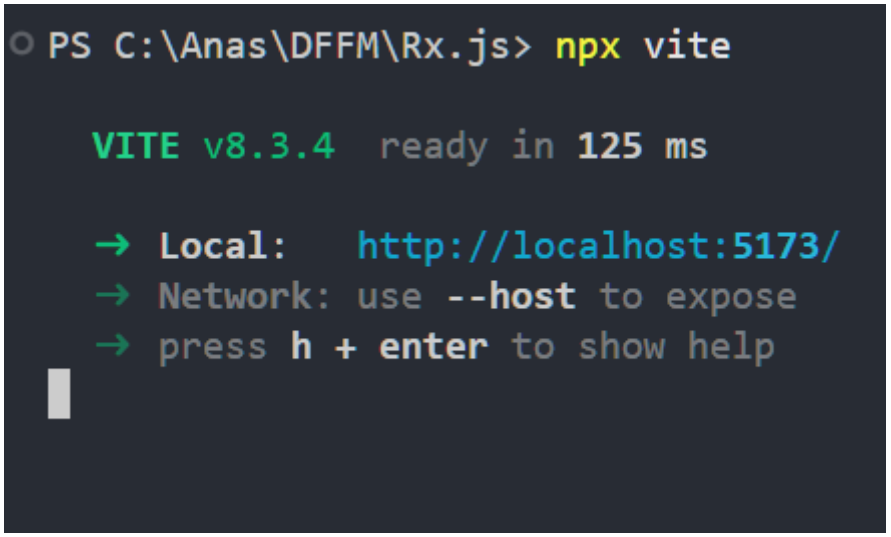
export default defineConfig({
  root: 'RelojPlanetario',
});
```

- En mi caso RelojPlanetario es la carpeta en donde tengo el pequeño proyecto, y dentro de esa carpeta tiene que estar si o si el index.html

- Y se tiene que quedar asi:



- Y ya esta, ahora solo tienes que poner el siguiente comando, copiar la ruta que te ponga y pegarla en el navegador:



4. Ejemplo básico de uso

- Al principio queria hacer una comparacion de relojes, uno en la Tierra y otro en Marte, pero estuve viendo un poco y resulta que la diferencia es casi nula, a no ser que pasen muchos meses o incluso años, no se notaria. Asi que he optado por hacer un reloj anual, y en vez de horas minutos y segundos sera años meses y dias.

- Al final, el proyecto basicamente consiste en una comparacion en cuanto tiempo tarda la tierra en dar una vuelta al Sol, y cuanto tardaria Marte si siguiese el mismo calendario gregoriano (1 año = 12 meses = 30 dias) y podria haber hecho un array para todos los meses, pero he decidido usar los mismos 30 dias en todos los meses, asi que al final 1 año serian 360 dias en la Tierra, pero como Marte tarda 687 dias, he hecho que su reloj vaya mas rapido (687 / 360) que seria unos 1.9 veces mas rapido que nuestro planeta (en realidad serian unos 1.888 mas rapido pero como he puesto que solo hayan 360 dias en un año lo he redondeado a 1.9).

- Funcionamiento:

    - El funcionamiento de la interfaz es basica, hay dos temporizadores y tres botones. Dependiendo del boton que pulses, ira mas rapido o mas lento (si pulsas el x10 ira 10 veces mas rapido, igual que el de x100, y si vuelves a pulsar el x1 ira normal otra vez)

- Fragmentos del codigo importantes:

    - En cuanto al codigo, he decidido usar typescript en vez de javascript (no cambia nada, es solo para practicarlo mas, de hecho RxJs esta basado en typescript pero como typescript esta basado en javascript pues da un poco igual lo que uses). Y no es mucho lo que hay que escribir de typescript o javascript nativo, la mayoria es de las librerias de RxJs, o bueno, mas bien la mitad o asi

    - Aun asi, quiero resaltar algunas piezas del codigo que al menos a mi me han costado de entender:

```
//Junta los clics de los tres botones en uno solo.
merge(
    fromEvent(bx1, 'click', () => {
        bx1.style.color = "salmon";          //Esto es simplemente para q
        bx1.style.borderColor = "salmon";

        bx10.style.color = "#FCF1D0";
        bx10.style.borderColor = "#FCF1D0";

        bx100.style.color = "#FCF1D0";
        bx100.style.borderColor = "#FCF1D0";

    }).pipe(map((): Velocidad => 1)),      //Velocidad normal
```

- merge(): ‘combina’ los tres observables en uno, asi, en vez de esperar la recepcion de tres botones (en mi caso) y despues crear un flujo de salida para cada uno, lo que hace es esperar por los tres botones a la vez, y cuando uno sea pulsado, ya hace lo que este dentro del respectivo evento del boton pulsado
- fromEvent(): es como un addEventListener, solo que lo implementa con el observable de rxjs, cada vez que llamas a esta funcion se crea un observable y se espera hasta que pase algo
- pipe(): es como un filtro, hay varios metodos que puedes meter dentro de el, en este caso el map()
- map(): sustituye lo que se ha filtrado en el pipe() por lo que tu quieras, en mi caso le he puesto que da igual el objeto que reciba, que lo que quiero que devuelva es que la velocidad ahora es x1, x10, o x100 (el evento simplemente sirve para cambiarle el color y saber que velocidad se quiere, si no no me hubiera servido para nada)

```
    }).pipe(map((): Velocidad => 100))    //100 veces mas rapido
).subscribe((v) => velocidad.next(v));    //Se guarda la velocidad
```

- El subscribe() es el que le dice al observable que se empieze a ejecutar. La v (v de velocidad) es el valor que sale o mas bien que devuelve, si en este caso hubiera pulsado el x100, la v valdria 100. Y el next(v) es simplemente para que velocidad tome el nuevo valor, o sea 100.

```
/**
 * Crea un reloj que cuenta días
 *
 * @param duracionDia Lo que dura un día en milisegundos
 * @returns Un flujo del total de días que han pasado
 */
function crearReloj(duracionDia: number): Observable<number> {
    return velocidad.pipe(
        //Si se cambia la velocidad, se para el contador anterior y en
        switchMap((v) => interval(duracionDia / v)),
        //Suma 1 día cada segundo
        scan((dias) => dias + 1, 0),
        //Empieza en 0 para que se vea "0 años, 0 meses, 0 días" desde
        startWith(0)
    );
}
```

- switchMap(): cancela el intervalo que estaba y lo sustituye por otro (si se ha pulsado el boton x10 entonces ese sera el que tenga el intervalo, hasta que otro boton sea pulsado)
- interval(): simplemente para decirle cuanto tiempo esperar entre ejecucion y ejecucion (si esta el boton x10 pulsado entonces se ejecutara 10 veces mas rapido)
- scan(): cada vez que que termina el interval() el scan va haciendo la operacion o mas vien ejecuta la funcion que tiene dentro, en este caso es simplemente ir aumentando en un dia a los dias que habian antes

```
//He puesto que un un segundo sea como un dia
crearReloj(1000).subscribe((dias) => (tiempoTierra.innerHTML = deDiasATexto(dias)));
```

- Recapitulando un poco lo visto anteriormente, aqui se llama a la funcion crearReloj y se le pasa la duracion de un dia (1000ms o 1s en este caso), despues el subscribe() le dice al observable que se empieze a ejecutar lo que haya dentro, y la funcion que esta dentro simplemente actualiza los dias y la vista para que se pueda ver el resultado:



Asi se veria al inicio de la ejecución

Y asi despues de un rato funcionando, si pulsas los botones x10 o x100 ira mas rapido



5. Ventajas, inconvenientes y conclusion

- Ventajas: Es mucho mas rapido de escribir que en javascript o typescript, ademas es mas eficiente que los metodos que incluyen por defecto, y en general una muy buena herramienta. Y ademas es muy facil de instalar y utilizar (aunque yo haya tenido problemas, llegandome a instalar vite, ha sido simplemente por el capricho de usar lo que he instalado, porque podrias perfectamente importar las librerias mediante urls y andarias de lujo)

- Desventajas: La mayor desventaja para mi es su legibilidad. Se que una vez que te acostumbres sera pan comido, como todo lo que haces, pero aunque tenga una base en la programacion en general, su sintaxis es algo que me ha costado entender y mucho mas acostumbrarme, y aparte de eso no creo que tenga muchas mas ventajas

- A mi personalmente me ha costado de entender (como ya he dicho antes), porque es una sintaxis con la que no habia trabajado antes, sin embargo, me ha parecido super interesante la tecnologia, y la programacion reactiva en general. Siento que para el usuario del dia a dia seria genial que las paginas web hagan uso normalmente de ella (si esque tiene sentido usarla). Puedo ver muchos usos en la programacion reactiva en general, pero especificamente RxJs, al estar construida en typescript, que a su vez esta hecho en base a javascript, que esta integrado en todos los navegadores, puede llegar a resultar muy util. Yo personalmente creo que vendria genial en sitios web que utilicen algoritmos continuamente, por ejemplo, las redes sociales, cada vez que pulses un video o le des like a una foto, que internamente se este generando y actualizando tu propio algoritmo constantemente. Y en general lo veo util para webs que requieran actualizacion en tiempo real. A pesar de haberme resultado un poco desafiante, sin duda indagare mas en este tipo de herramientas una vez terminado mis estudios principales, porque veo increible su capacidad de adaptabilidad.

6. Recursos y bibliografia

- https://rxjs.dev/guide/installation
- https://rxjs-dev.firebaseapp.com/guide/overview
- https://www.youtube.com/playlist?list=PL55RiY5tL51pHpagYcrN9ubNLVXF8rGVi (no lo he utilizado casi nada porque es un poco antiguo y muchas de las funciones estan deprecated)
- https://nodejs.org/en/download